

# Documento de Integração o Rota Fácil

[Documento de Integração: Front-end \(Flutter\) e Back-end \(API\) — Rota Fácil](#)

[▮ PARTE 1: Mapa de Telas e Funcionalidades](#)

[⚙️ PARTE 2: Estrutura de Dados](#)

[👤 PARTE 3: Autenticação](#)

[🏠 PARTE 4: Referência Rápida de Endpoints](#)

## Documento de Integração: Front-end (Flutter) e Back-end (API) — Rota Fácil

**Atualizado em relação à versão anterior.** O backend passou por uma refatoração completa (arquitetura de Blueprints + RBAC), e desde a última revisão ganhou busca por proximidade geográfica (coordenadas de trilha + busca por raio a partir do GPS do usuário) e vínculo entre imagem e trilha. Os campos e regras abaixo refletem o estado real da API — o Flutter precisa seguir exatamente isso, não versões anteriores deste documento.

### ▮ PARTE 1: Mapa de Telas e Funcionalidades

Pasta sugerida: `lib/screens/`

#### 1. Módulo de Autenticação e Perfil

- `login_screen.dart` : Tela com E-mail e Senha. Ao autenticar, a API retorna um **token JWT** que precisa ser guardado localmente (ex: `flutter_secure_storage`) e reenviado em toda chamada protegida (ver Parte 3).
- `cadastro_screen.dart` : Formulário de registro (campos na Parte 2-A).
- `perfil_screen.dart` : Visualizar e editar os próprios dados.

#### 2. Módulo de Exploração (Visão do Turista/Usuário)

- `home_feed_screen.dart` : Lista de trilhas disponíveis (cards).
- `busca_filtros_screen.dart` : Busca avançada por Cidade, Estado, Dificuldade, Período e Proximidade (Parte 2-C).
- `detalhes_trilha_screen.dart` : Detalhes completos ao clicar em um card.
- `meus_agendamentos_screen.dart` : Histórico de agendamentos do usuário logado (Parte 2-D).

#### 3. Módulo de Gestão (Visão do Guia / Agência)

O backend bloqueia usuários com papel `"usuario"` nessas rotas — apenas `"guia"` e `"agencia"` têm acesso.

- `painel_guia_screen.dart` : Trilhas criadas pelo guia/agência logado.
- `formulario_trilha_screen.dart` : Criar/editar trilha (campos na Parte 2-B).

### ⚙️ PARTE 2: Estrutura de Dados

## A. Cadastro de Usuário — `POST /api/usuarios/`

Rota **pública**, não exige token.

| Campo    | Tipo   | Obrigatório              | Observação                                                                |
|----------|--------|--------------------------|---------------------------------------------------------------------------|
| nome     | String | Sim                      |                                                                           |
| email    | String | Sim                      |                                                                           |
| senha    | String | Sim                      |                                                                           |
| telefone | String | Não                      |                                                                           |
| cidade   | String | Não                      |                                                                           |
| estado   | String | Não                      |                                                                           |
| tipo     | String | Não (default "usuario" ) | <b>Dropdown com exatamente estas opções:</b> "usuario", "guia", "agencia" |

⚠ **O nome do campo é `tipo`, não `tipo_perfil`.** E o valor para turista/cliente comum é "usuario", não "turista" — mandar "turista" ou qualquer valor fora da lista resulta em erro 400.

O papel "admin" não pode ser atribuído por esse formulário — é bloqueado propositalmente pelo backend por segurança.

## B. Criação de Trilha — `POST /api/trilhas/`

Rota **protegida**: exige header `Authorization: Bearer <token>` de um usuário com `tipo = "guia" ou "agencia"`.

| Campo          | Tipo   | Obrigatório               | Observação                                                                                                                                                             |
|----------------|--------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nome           | String | Sim                       |                                                                                                                                                                        |
| descricao      | String | Sim                       |                                                                                                                                                                        |
| dificuldade    | String | Sim                       | Sugestão de Dropdown: "facil", "media", "difícil"                                                                                                                      |
| cidade         | String | Não                       |                                                                                                                                                                        |
| estado         | String | Não                       | Sigla (ex: PR, SP)                                                                                                                                                     |
| modalidade     | String | Não (default "Trekking" ) |                                                                                                                                                                        |
| distancia_km   | Number | Não (default 0.0 )        | Esse é o comprimento do percurso da trilha (ex: "8.5 km de trilha"), <b>não confundir</b> com a distância até o usuário que aparece na busca (Parte 2-C)               |
| data_atividade | String | Não                       | <b>Envie no formato curto AAAA-MM-DD</b> (não é ISO com hora). Precisa desse formato porque o backend compara essas datas como texto nos filtros de busca por período. |
| latitude       | Number | Não                       | Ponto no mapa selecionado na tela de criação (toque no mapa, não digitado)                                                                                             |

|            |        |     |                                                                                             |
|------------|--------|-----|---------------------------------------------------------------------------------------------|
| longitude  | Number | Não | Idem                                                                                        |
| imagem_url | String | Não | URL retornada pelo endpoint de upload (Parte 2-E) — enviar depois de fazer o upload da foto |

Latitude e longitude são **opcionais, mas os dois juntos** — se enviar um, envie o outro também. Trilhas sem coordenada continuam funcionando normalmente em todas as telas, só não aparecem em buscas por proximidade (Parte 2-C) nem podem ter marcador no mapa de resultados.

✓ **ponto\_encontro foi removido definitivamente** — decisão tomada pelo time: as coordenadas ( latitude / longitude ) já cobrem a necessidade de indicar o local de encontro/partida da trilha no mapa. Se a tela do Flutter ainda tiver esse campo, pode remover — ele nunca foi persistido e não vai voltar.

O campo id\_guia é preenchido automaticamente pelo backend com o e-mail de quem criou a trilha, a menos que seja explicitamente enviado outro valor no corpo da requisição.

## C. Busca Avançada de Trilhas — GET /api/trilhas/busca

Rota protegida (qualquer usuário autenticado). Parâmetros via **Query String**:

- ?cidade=Maringá
- ?estado=PR
- ?dificuldade=media
- ?data\_inicio=2026-09-01&data\_fim=2026-09-30 (formato curto AAAA-MM-DD)
- ?latitude=-27.5954&longitude=-48.5480&raio\_km=20 (busca por proximidade — ver detalhes abaixo)

Todos os filtros são opcionais e combináveis entre si. A busca só retorna trilhas com ativo: true.

### Busca por proximidade ( latitude + longitude + raio\_km )

Os três parâmetros precisam vir **juntos** — se faltar algum, o backend ignora os três e trata como busca normal (sem filtro de distância).

**Origem do ponto de busca: GPS do dispositivo.** O Flutter deve capturar a localização atual do usuário (via pacote geolocator, tratando os estados de permissão negada/permitida) e enviar como latitude / longitude. Não é endereço digitado nem localização salva no perfil — decisão tomada pra manter o escopo simples dentro do prazo da competição. Pode evoluir pra isso depois, mas hoje o contrato da API já está pronto pra aceitar qualquer origem que o Flutter mandar.

raio\_km é escolhido pelo usuário na tela (ex: slider ou dropdown com opções como 5 / 20 / 50 / 100 km) — o backend aceita qualquer número positivo, não há valor fixo.

Quando a busca por proximidade é usada, cada trilha no resultado ganha um campo extra:

```
{
  "id": "...",
  "nome": "Trilha do Pico",
  "latitude": -27.60,
  "longitude": -48.55,
  "distancia_usuario_km": 3.42,
  ...
}
```

distancia\_usuario\_km é a distância real (linha reta) entre o usuário e a trilha — os resultados já vêm **ordenados do mais próximo pro mais distante**. Não confundir com distancia\_km (Parte 2-B), que é o tamanho do percurso da trilha em si.

⚠ **Combinação com filtro de data:** funciona, mas com uma limitação técnica do banco de dados — quando a busca é por proximidade, o backend não consegue aplicar o filtro de data como índice otimizado (faz por cima dos resultados já filtrados por localização). Na prática o resultado final está

sempre correto, só não é tão eficiente quanto poderia ser. Não exige nada diferente do Flutter, é só um detalhe de implementação interna.

⚠ **Trilhas cadastradas antes dessa funcionalidade** podem não aparecer em buscas por proximidade até serem editadas com uma coordenada (ou até o backend rodar uma migração pontual) — se durante os testes uma trilha antiga “sumir” só da busca por raio (mas aparecer normalmente nas outras buscas), é esse o motivo, não é bug.

### D. Agendamentos

**Criar agendamento** — `POST /api/agendamentos/` Protegida. Papéis aceitos: `"usuario"`, `"agencia"`, `"guia"`.

| Campo                      | Tipo   | Obrigatório                                                                  |
|----------------------------|--------|------------------------------------------------------------------------------|
| <code>id_trilha</code>     | String | Sim (precisa ser um ID de trilha existente, senão retorna <code>404</code> ) |
| <code>data_agendada</code> | String | Sim                                                                          |
| <code>valor_pago</code>    | Number | Sim                                                                          |
| <code>id_guia</code>       | String | Não                                                                          |

`id_usuario` **não deve ser enviado pelo Flutter** — o backend preenche automaticamente com o ID de quem está logado, por segurança.

**Listar meus agendamentos** — `GET /api/agendamentos/usuario/<id_usuario>` Protegida. Só retorna dados se `id_usuario` na URL for o mesmo do usuário logado (ou se for admin).

**Cancelar agendamento** — `PUT /api/agendamentos/<id_agendamento>/cancelar` Protegida, sem corpo. Só quem criou o agendamento, o guia responsável, ou um admin podem cancelar.

### E. Upload de Imagem — `POST /api/uploads/imagem`

Protegida (qualquer usuário autenticado). Envio via **multipart/form-data**, com a imagem na chave `image` (não `imagem`, em inglês mesmo).

Resposta em caso de sucesso:

```
{
  "mensagem": "Upload realizado com sucesso!",
  "url": "https://i.ibb.co/xxxxx/foto.jpg"
}
```

✓ **Vínculo com trilha resolvido:** essa `url` deve ser enviada como o campo `imagem_url` no `POST /api/trilhas/` (criação) ou `PUT /api/trilhas/<id>` (edição). Fluxo completo: (1) usuário escolhe a foto → (2) `POST /api/uploads/imagem` → (3) pega a `url` da resposta → (4) inclui como `imagem_url` ao criar ou editar a trilha.

## 🔒 PARTE 3: Autenticação

**Login** — `POST /api/auth/login`

```
{
  "email": "usuario@exemplo.com",
  "senha": "123456"
}
```

Resposta:

```
{
  "mensagem": "Autenticação bem-sucedida",
  "nome": "Nome do Usuário",
  "tipo": "usuario",
  "token": "eyJhbGciOi..."
}
```

O campo `tipo` retornado ajuda o Flutter a decidir qual tela mostrar (ex: `painel_guia_screen` só se `tipo` for `"guia"` ou `"agencia"`).

**Em toda chamada a uma rota protegida**, adicionar o header:

```
Authorization: Bearer <token>
```

O token expira em **2 horas**. Se uma chamada retornar `401` com mensagem de token expirado, a tela deve redirecionar pro login novamente.

## 📁 PARTE 4: Referência Rápida de Endpoints

| Método             | Rota                                            | Protegida? | Papéis permitidos                                 |
|--------------------|-------------------------------------------------|------------|---------------------------------------------------|
| POST               | /api/usuarios/                                  | Não        | —                                                 |
| GET                | /api/usuarios/                                  | Sim        | admin                                             |
| GET / PUT / DELETE | /api/usuarios/<id>                              | Sim        | dono ou admin                                     |
| POST               | /api/auth/login                                 | Não        | —                                                 |
| POST               | /api/trilhas/                                   | Sim        | guia , agencia                                    |
| GET                | /api/trilhas/                                   | Sim        | qualquer autenticado                              |
| GET                | /api/trilhas/<id>                               | Sim        | qualquer autenticado                              |
| GET                | /api/trilhas/busca                              | Sim        | qualquer autenticado                              |
| GET                | /api/trilhas/periodo?<br>data_inicio=&data_fim= | Sim        | qualquer autenticado                              |
| PUT / DELETE       | /api/trilhas/<id>                               | Sim        | criador, guia responsável, ou admin               |
| POST               | /api/agendamentos/                              | Sim        | usuario , agencia , guia                          |
| GET                | /api/agendamentos/usuario/<id>                  | Sim        | dono ou admin                                     |
| GET                | /api/agendamentos/guia/<id>                     | Sim        | guia , agencia , admin (dono ou admin / agencia ) |
| PUT                | /api/agendamentos/<id>/cancelar                 | Sim        | dono, guia responsável, ou admin                  |
| POST               | /api/uploads/imagem                             | Sim        | qualquer autenticado                              |

Base URL local: `http://127.0.0.1:5000` (ajustar para o endereço de deploy quando disponível).